Xensible expert curriculum companion

Advanced Operator / Codex Track

Advanced training for power users who want to supervise Codex and similar local full-control systems with judgment, testing, and rollback habits.

Format	Private coaching or cohort sessions with local practice, repo walkthroughs, and verification labs.
Level	Level 4: advanced operator and local-agent supervision
Audience	Technically curious professionals, builders, content-system owners, and advanced AI users ready for local controlled environments.

Learner outcomes

- Understand what local full-control systems can access and why that changes the supervision model.
- Use repo inspection, file edits, tests, browser checks, Git commits, and deployment review responsibly.
- Develop rollback thinking so AI-assisted work remains reversible, inspectable, and owned by the human.
- Operate Codex-style systems with explicit boundaries, verification habits, and human approval points.

Guiding questions

- What changes when an AI system can read files, run commands, inspect browsers, and edit projects?
- Which actions are safe for an agent to take, and which require explicit human approval?
- How will we verify outputs with tests, screenshots, diffs, and deployment checks?
- How will we recover if an AI-assisted change is wrong?

Readiness checks

- The operator can explain what local files, commands, browsers, and deployments the agent can affect.
- The operator can request scoped changes and inspect diffs before accepting them.
- The operator can run appropriate verification before committing or publishing.
- The operator has a rollback path and knows when to stop for human review.

Curriculum modules

From chatbot to local operator

We compare ordinary LLM sessions with local agents that can read files, run commands, inspect browsers, and edit projects.

Supervising code and content changes

Participants learn to ask for repo inspection, scoped edits, diffs, tests, and clear summaries before trusting changes.

Full-control safety boundaries

We define where local access helps, where it becomes risky, and what should require explicit human approval.

Deployment and rollback practice

Participants practice commit hygiene, public-site checks, deployment verification, and recovery paths.

Operator judgment

Advanced users learn when to let the agent proceed, when to require confirmation, when to inspect manually, and when to stop.

Session flow

Orient

Compare chat-based AI with local agent systems that can act inside a project environment.

Inspect

Read project structure, current state, dependencies, and risk before asking for changes.

Constrain

Write scoped instructions that define files, boundaries, verification, and approval points.

Verify

Use linting, tests, screenshots, diffs, and manual review to decide whether work is acceptable.

Recover

Practice commit hygiene, rollback thinking, and post-deployment checks.

Practice labs

Repo inspection lab

Ask the agent to inspect a project and produce a change plan before editing anything.

Scoped edit lab

Make a small content or UI change, then review the diff and reject unrelated churn.

Verification lab

Run lint, build, browser screenshots, and content checks before deciding whether to commit.

Rollback lab

Use Git history and deployment checks to reason about how to recover from a bad change.

Approval boundary lab

Classify actions such as file edits, package installs, API calls, credential use, deployment, and deletion by approval level.

Materials to develop

- Local agent supervision checklist
- Git and deployment primer
- Verification lab worksheet
- Rollback and recovery checklist
- Approval-boundary matrix
- Codex prompt patterns for scoped work

Diagram candidates

These are intentionally left as candidate visuals until diagram parameters are chosen.

- Agent control loop: inspect, edit, test, review, commit
- Local full-control safety boundary map
- Approval boundary matrix for local agent actions

Follow-up work

- Run one scoped agent-assisted change on a low-risk project.
- Capture the prompt, diff, checks, and final decision in a practice log.
- Build a personal checklist for future Codex-style work before using it on higher-risk projects.

Session design reminders

- Keep examples non-sensitive and realistic.
- Emphasize education, judgment, review, and careful experimentation.
- Do not promise legal, medical, compliance, cybersecurity, or automation outcomes.
- Name when a stronger tool, private environment, or policy decision is needed before proceeding.

API and tool boundary

This curriculum companion does not require an API call. Any secure password protection, private client portal, or live AI workflow exercise that transmits data to an external model should be reviewed before implementation.